

PATRONES DE DISEÑO

OBJETIVOS

- ① Conocer cuáles son los patrones de diseño creacionales
- ① Identificar cuándo aplicar cada uno de los patrones de diseño creacionales.

Dennis S. Cohn Muroy, Mag. Ing.
Pontificia Universidad Católica del Perú



1

PATRONES DE DISEÑO CREACIONALES

Detallemos algunos de los patrones ...

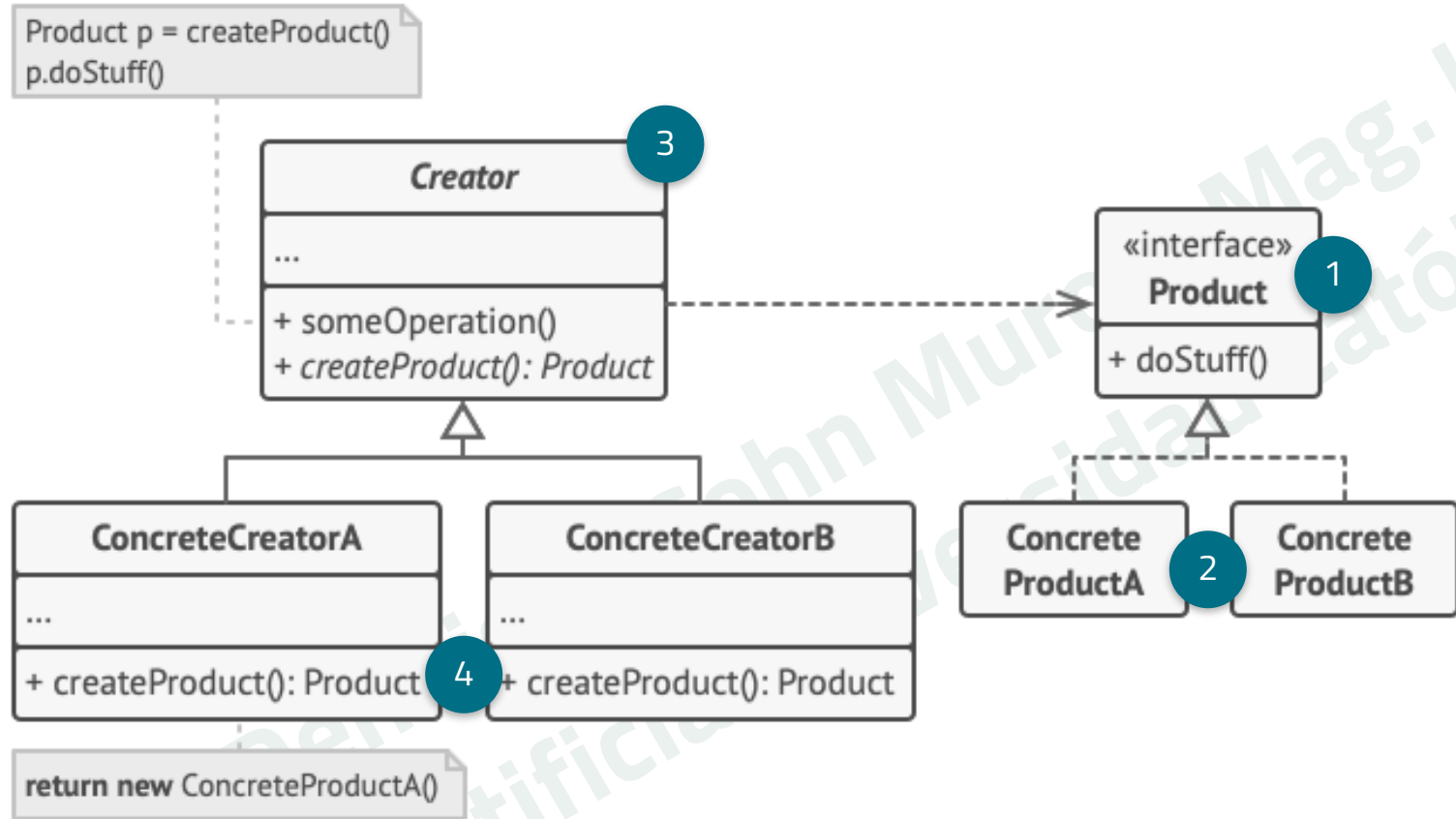
PATRÓN: FACTORY METHOD

PROBLEMA QUE CUBRE

- Los *frameworks* necesitan estandarizar el modelo de la arquitectura para todas las aplicaciones; pero deben permitir la definición de objetos para dominios particulares.

¿CÓMO LO HACE?

Define una interfaz para crear un objeto; pero las subclasses deciden que clase se instanciará.



- 1 Interfaz de Producto: común a todos los objetos que puede producir.
- 2 Productos concretos: implementaciones de la interfaz Producto.
- 3 Clase Creadora Abstracta: declara el método que devuelve nuevos objetos de producto.
- 4 Creadores Concretos: devuelven un tipo particular de producto.

CONSECUENCIAS: FACTORY METHOD

- ⦿ (+) Separa el código de creación del producto del código que lo utiliza. Esto facilita la extensibilidad de nuevos productos.
- ⦿ (+) El Factory Method puede optar por devolver un objeto existente en lugar de crear uno nuevo.

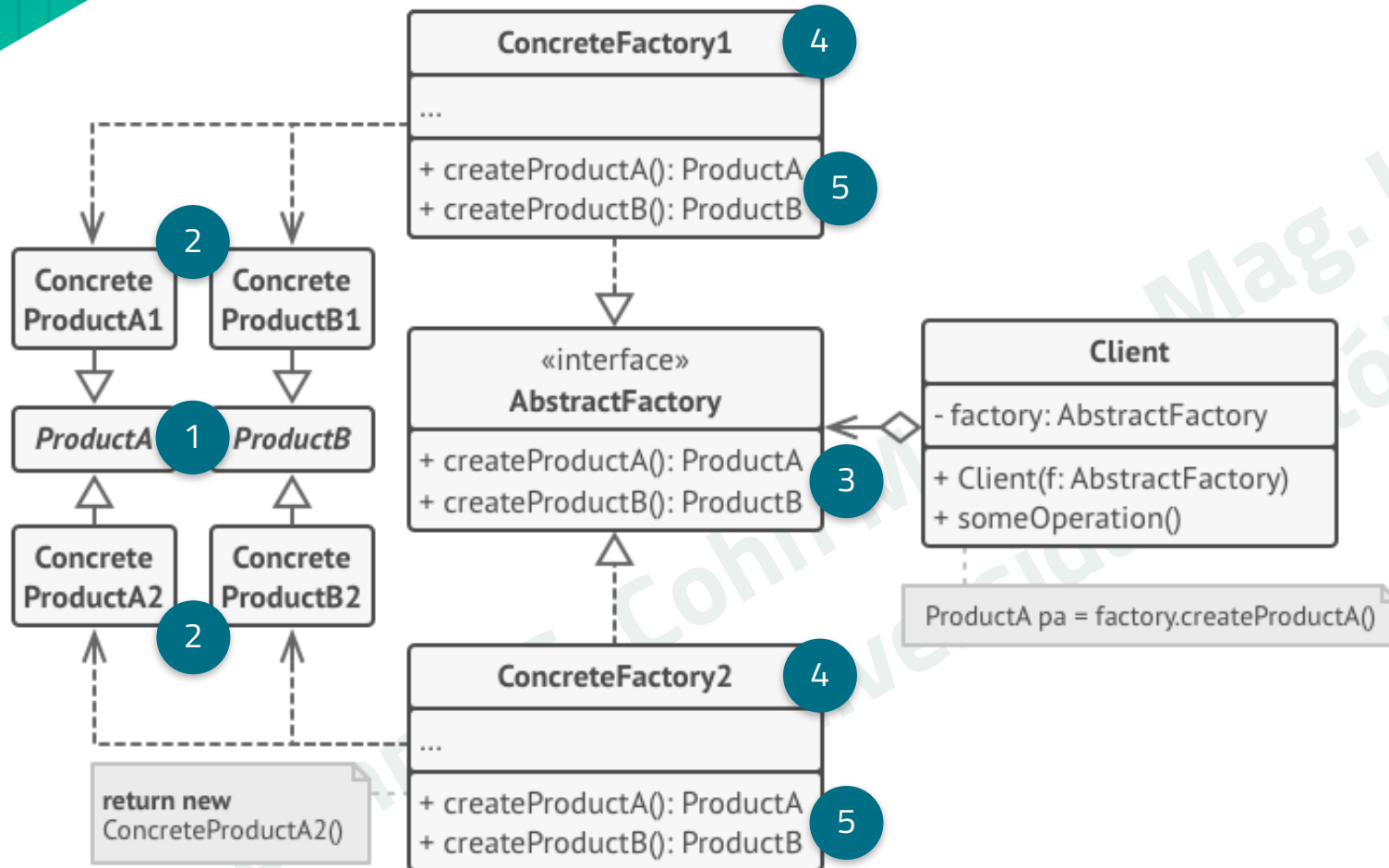
PATRÓN: ABSTRACT FACTORY

PROBLEMA QUE CUBRE

- ⦿ Garantizar extensibilidad y portabilidad de la solución.
- ⦿ Evita el uso de condiciones anidadas para instanciar clases.

¿CÓMO LO HACE?

Permite aprovisionar una familia de objetos relacionados sin necesidad de especificar sus clases concretas.



- 1 Clase Abstracta por tipo de Productos.
- 2 Implementar colecciones de Productos por cada tipo.
- 3 Interfaz de la Factoría que declara los métodos para instanciar cada tipo de producto.
- 4 Implementación de la Factoría por cada colección.
- 5 Es importante que se retorne la clase abstracta correspondiente.

CONSECUENCIAS: ABSTRACT FACTORY

- ⦿ (+) Encapsula las clases concretas; limita los tipos de objetos que una aplicación crea.
- ⦿ (+) Facilita el intercambio de las familias de productos; con solo el intercambio del ConcreteFactory.
- ⦿ (+) Promueve consistencia entre productos a través de la creación de una familia de productos.
- ⦿ (-) Es complejo agregar nuevos tipos de productos; ello implica modificar el AbstractFactory y sus clases hijas.

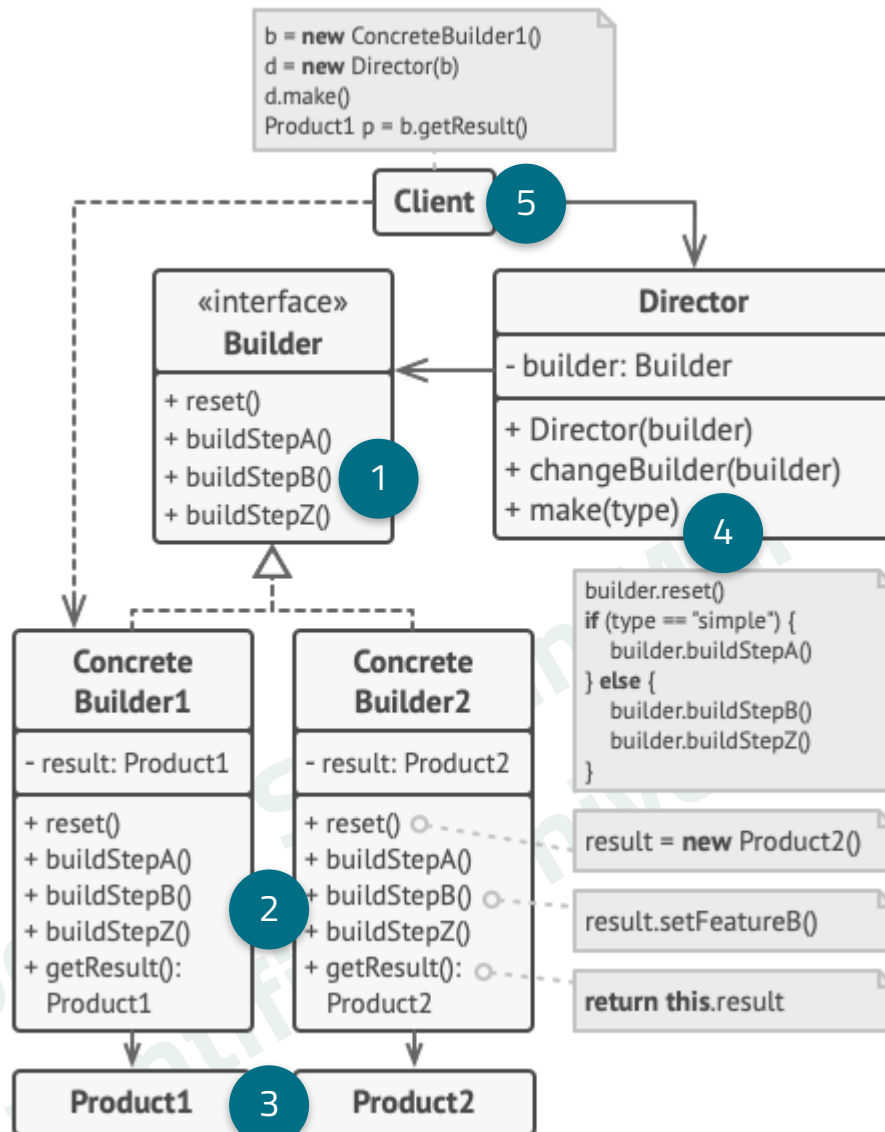
PATRÓN: BUILDER

PROBLEMA QUE CUBRE

- Cómo construir objetos complejos paso a paso.

¿CÓMO LO HACE?

Organiza la construcción de objetos en una serie de pasos. Para crear un objeto, se ejecuta una serie de estos pasos desde un objeto constructor.



- 1 Interfaz Constructora declara pasos de construcción comunes.
- 2 Constructores Concretos implementan los pasos de construcción.
- 3 Los Productos son los objetos resultantes. No heredan de los Constructores
- 4 Clase Directora define el orden de los pasos de construcción
- 5 El Cliente define el Constructor a utilizar.

CONSECUENCIAS: BUILDER

- ⦿ (+) Abstrae la estructura del producto y los pasos que se siguen para su ensamblado.
- ⦿ (+) Control granular sobre los pasos a seguir para el proceso de construcción.
- ⦿ (+) Evita sobrecargar o complejizar el método constructor.

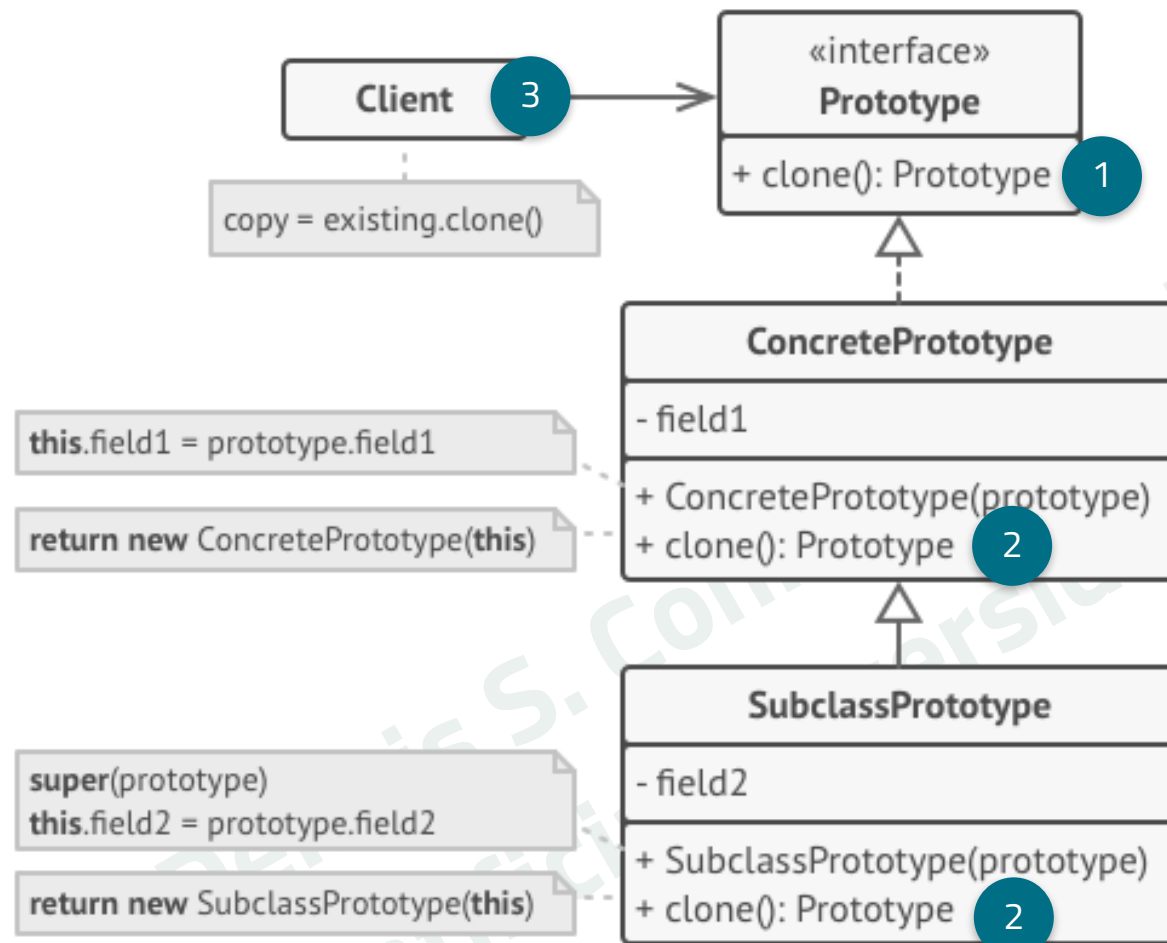
PATRÓN: PROTOTYPE

PROBLEMA QUE CUBRE

- Copiar objetos existentes sin conocer la estructura interna del objeto.

¿CÓMO LO HACE?

Delega el proceso de clonación a los propios objetos que están siendo clonados. El patrón declara una interfaz común para todos los objetos que soportan la clonación.



- 1 Interfaz Prototipo declara los métodos de clonación.
- 2 Prototipo Concreto implementa el método de clonación. Copia la información del objeto original al clon.
- 3 Cliente puede producir una copia de cualquier objeto que siga la interfaz del prototipo.

CONSECUENCIAS: PROTOTYPE

- ⦿ (+) Permite incorporar una nueva clase concreta con solo registrar una nueva instancia.
- ⦿ (+) No se requiere una clase Factoría. Se pueden instanciar nuevos objetos en base a la clonación.
- ⦿ (-) Complejidad al clonar objetos con referencias circulares.

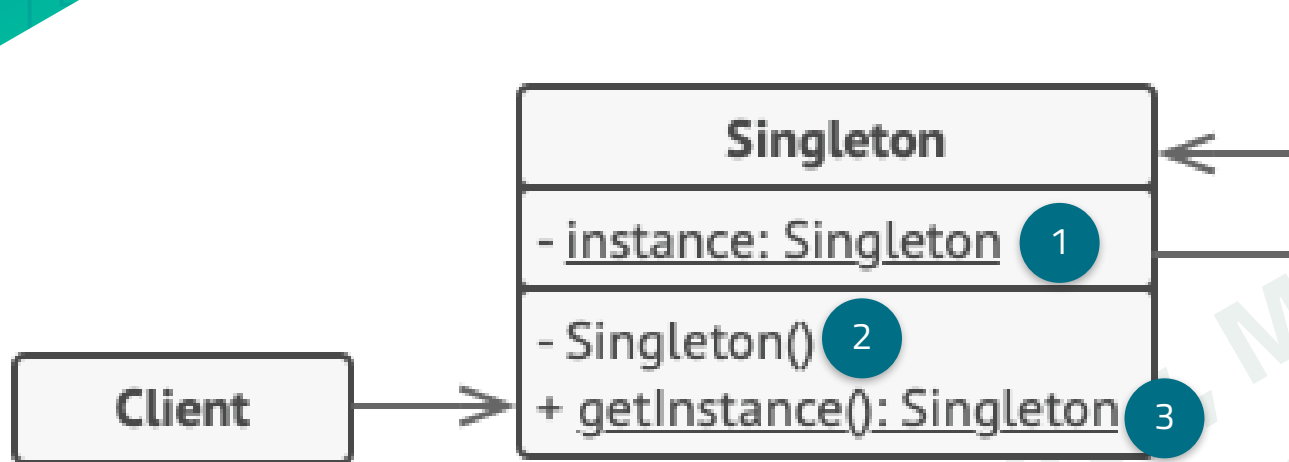
PATRÓN: SINGLETON

PROBLEMA QUE CUBRE

- ⦿ Punto de acceso único para un recurso.
- ⦿ Acceso global inmutable.

¿CÓMO LO HACE?

Asegura que una clase tiene una única instancia.



- 1 Instancia estática y privada de la misma clase.
- 2 Constructor privado
- 3 Método estático y público para obtener acceso a la instancia

```
if (instance == null) {  
    // Note: if you're creating an app with  
    // multithreading support, you should  
    // place a thread lock here.  
    instance = new Singleton()  
}  
return instance
```

CONSECUENCIAS: SINGLETON

- ⦿ (+) Acceso controlado a una sola instancia.
- ⦿ (+) Puede ajustarse para soportar un número máximo de instancias por objeto.
- ⦿ (-) Puede enmascarar un mal diseño.

Dennis S. Cohn Muroy, Mag. Ing.
Pontificia Universidad Católica del Perú

2

REFERENCIAS

BIBLIOGRAFÍA

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design patterns: elements of reusable object-oriented software. Pearson Deutschland GmbH.
- Freeman, E., & Robson, E. (2020). Head First Design Patterns. O'Reilly Media.
- Design patterns and refactoring. (n.d.). https://sourcemaking.com/design_patterns

Créditos:

- Plantilla de la presentación por [SlidesCarnival](#)
- Fotografías por [Unsplash](#)
- Diseño del fondo [Hero Patterns](#)